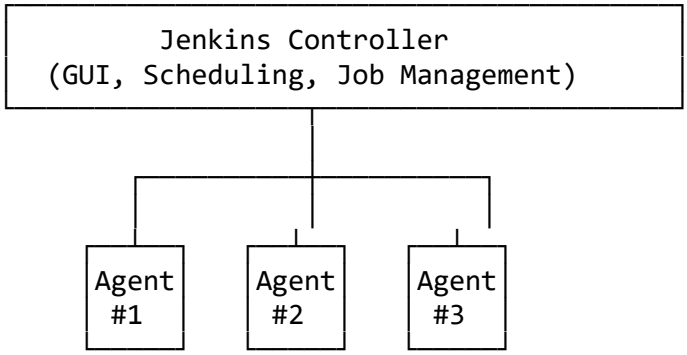
GitHub Actions

Foundations & Architecture

1.1 GitHub Actions vs Jenkins: Quick Reference

What You Know from Jenkins

Jenkins Architecture:



Jenkins Pipeline Structure:

```
pipeline {
  agent any
  stages {
    stage('Build') { ... }
    stage('Test') { ... }
    stage('Deploy') { ... }
  }
}
```

1.2 Concept Mapping: Jenkins → GitHub Actions

Jenkins Concept	GitHub Actions Equivalent	Purpose
Job	**Workflow**	Top-level automation unit
Pipeline	**Workflow**	Declarative pipeline definition
Stage	**Job**	Logical grouping of steps
Step	**Step**	Individual command or action
Plugin	**Action**	Reusable task from marketplace
Agent	**Runner**	Machine executing the job
Credential	**Secret**	Protected environment variable

Jenkins Concept	GitHub Actions Equivalent	Purpose
Parameter	**Input**	User-provided or context value
Trigger	**Event (on:)**	What causes the workflow to run

1.3 Core Definitions

Workflow

A YAML file in `.github/workflows/` that defines an automated process. It contains: - **Triggers** (events): when to run - **Jobs**: logical units of work - **Steps**: individual commands or actions

File Location: `.github/workflows/my-workflow.yml`

Minimum YAML Structure:

```yaml
name: My First Workflow
on: push
jobs:
  my-job:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello GitHub Actions!"
```

Job

A set of steps executed on the same runner. Key characteristics: - Jobs run **in parallel** by default (unlike Jenkins stages) - Can have dependencies using `needs:` - Run on a specific `runs-on:` runner - Share outputs via `outputs:`

Example:

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Build job"

  test:
    needs: build  # Waits for 'build' to complete
    runs-on: ubuntu-latest
    steps:
      - run: echo "Test job"
```

Step

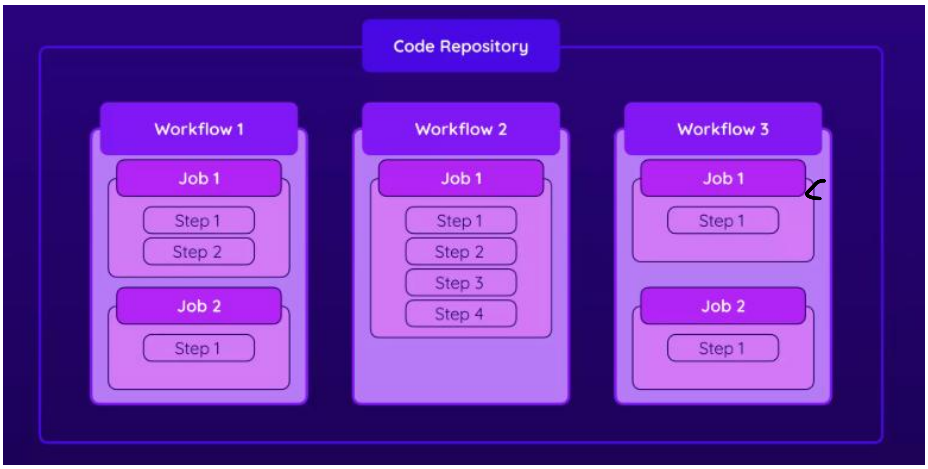
The smallest unit of work. Can be: 1. **Shell command** (`run:`) 2. **Action** (`uses:`) 3. **Conditional script** with `if:`

Examples:

```yaml
steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Run tests
    run: npm test

  - name: Deploy (only on main)
    if: github.ref == 'refs/heads/main'
    run: ./deploy.sh
```



Action

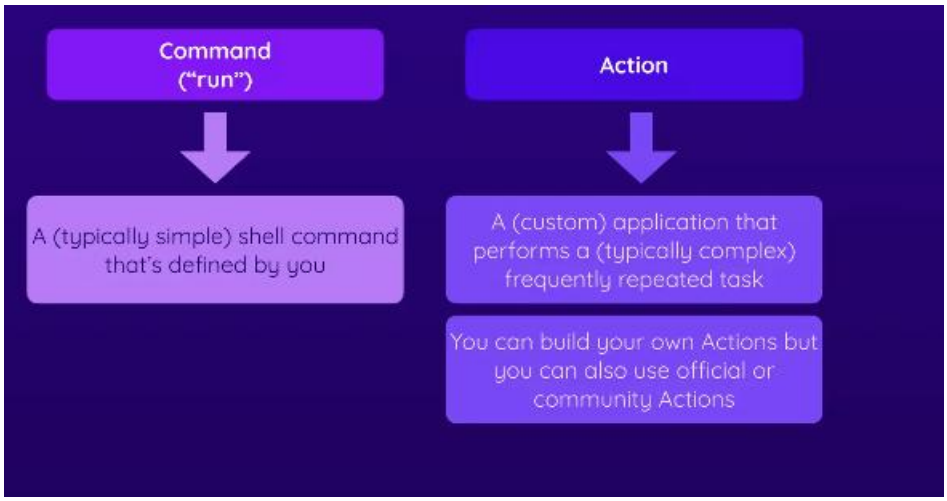
Reusable module performing a specific task. Types: 1. **Docker action**: runs in container 2. **JavaScript action**: runs on Node.js 3. **Composite action**: combines steps

Usage:

```yaml
uses: actions/checkout@v4
# Format: owner/repo@version or owner/repo/path@version
```

Common Actions:

Action	Purpose
actions/checkout@v4	Clone repository code
actions/setup-node@v4	Set up Node.js environment
actions/upload-artifact@v4	Store build artifacts
actions/download-artifact@v4	Retrieve stored artifacts



Runner

A machine (hosted or self-hosted) that executes jobs.

GitHub-Hosted Runners:

```yaml
runs-on: ubuntu-latest      # Linux
runs-on: windows-latest     # Windows
runs-on: macos-latest       # macOS
```

Self-Hosted Runners:

```yaml
runs-on: [self-hosted, linux, x64, gpu]  # Labels you define
```

Events (Triggers)

GitHub events that start workflows.

Common Events:

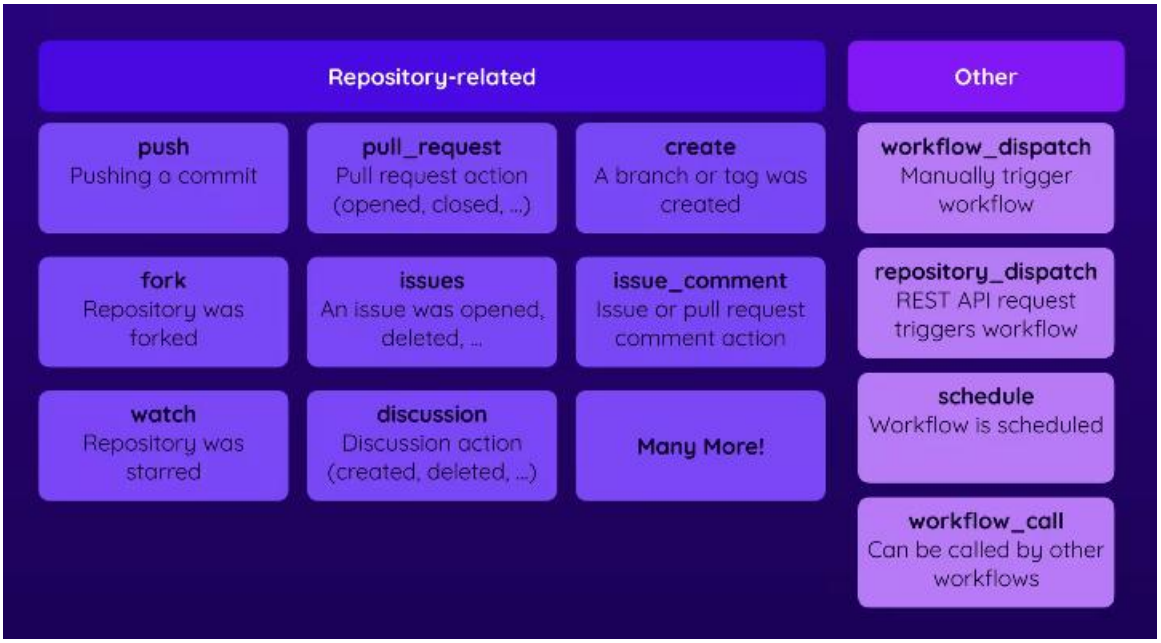
```yaml
on:
  push:
    branches: [main, develop]
    paths: ['src/**', 'package.json']

  pull_request:
    branches: [main]

  schedule:
    - cron: '0 9 * * 1'  # Every Monday at 9 AM UTC

  workflow_dispatch:  # Manual trigger via UI

  release:
    types: [published]
```



1.4 Architecture Diagram: End-to-End Flow

```
Developer pushes code to GitHub
          │
          ↓
GitHub detects event (push, PR, schedule)
          │
          ↓
Workflow triggered (.github/workflows/ci.yml)
          │
          ├─→ Job 1: Build (ubuntu-latest runner)
          │    ├─ Checkout code
          │    ├─ Install dependencies
          │    ├─ Compile/build
          │    └─ Upload artifact
          │
          ├─→ Job 2: Test (ubuntu-latest runner)
          │    ├─ Download artifact
          │    ├─ Run unit tests
          │    ├─ Run integration tests
          │    └─ Generate test report
          │
          └─→ Job 3: Deploy (Ubuntu runner, needs: [build, test])
               ├─ Download artifact
               ├─ Deploy to production
               └─ Send Slack notification

All logs available in GitHub UI → Actions tab
```

Key Learnings:

Workflow files live in `.github/workflows/`
YAML syntax and proper indentation matter
Logs are visible in GitHub UI
`workflow_dispatch` allows manual triggers
Multiple events can trigger the same workflow

1.6 Summary and Key Takeaways

Concept	Key Point
Workflow	YAML file that defines the entire automation
Jobs	Run in parallel; ordered with `needs:`
Steps	Individual tasks (shell commands or actions)
Actions	Reusable marketplace modules
Runners	Machines executing workflows (GitHub-hosted or self-hosted)
Events	GitHub events triggering workflows (push, PR, schedule)
Jenkins vs Actions	SCM-native, distributed, no controller server needed

---